

DIÁRIO DE BORDO: Sistema de Agendamento de Consultas em Clínica Universitária

1. Relato Semanal de Desenvolvimento

Semana 1: Planejamento e Estruturação de Dados

Nesta primeira semana, o foco foi compreender os requisitos do sistema e desenhar a arquitetura dos dados. Decidimos modularizar o código separando as responsabilidades em ficheiros .h e .c (paciente, agendamento e lista). Implementamos a Lista Simplesmente Encadeada para o registo de pacientes e definimos as structs principais. O maior desafio inicial foi garantir que a lógica de negócio fizesse sentido (ex: um paciente não pode ser cadastrado duas vezes com o mesmo CPF).

Semana 2: Implementação da Lista com Cabeçalho e Interfaces

Na segunda semana, avançamos para o coração do sistema: os agendamentos. Implementamos a Lista com Cabeçalho (Nó Sentinela) exigida para o histórico. Focámo-nos em desenvolver as funções de interface (menuCadastrarAgendamento, menuRemoverAgendamento, etc.), garantindo a comunicação correta entre a lista de pacientes e a de agendamentos. Realizamos testes de inserção e remoção, corrigindo bugs relacionados com a limpeza do buffer do teclado e a manipulação de strings (strcpy). Por final demos os toques finais no main para com clareza apresentar ao usuario a funcionalidade como um todo.

2. Divisão de Tarefas (Quem fez o quê)

[Jean Kenold Dorce]: Responsável pela implementação das funções de interface no agendamento.c (cadastro, listagem e remoção), validação de entradas (evitando buffer overflow com scanf limitados) e comunicação cruzada entre a lista de pacientes e a lista de agendamentos.

2.1 Dificuldades Encontradas e Soluções

Dificuldade 1: "Saltos" na leitura de dados (Buffer do Teclado).

Problema: Ao ler um número inteiro (como o número da sala) e logo a seguir ler uma string (como a data), o programa "saltava" a pergunta da data porque o \n (Enter) ficava no buffer de memória.

Solução: Implementamos a limpeza de buffer com um ciclo while (getchar() != '\n'); logo após a leitura do inteiro.

Dificuldade 2: Remoção segura na Lista de Agendamentos.

Problema: Remover um nó no meio ou no início de uma lista dinâmica pode causar a perda de referência e falhas de segmentação (Segmentation Fault).

Solução: A utilização da Lista com Cabeçalho facilitou o processo. Criamos ponteiros anterior (apontando para o header) e atual (apontando para header->proximo). Assim, a lógica de religar os nós (anterior->proximo = atual->proximo) funcionou perfeitamente sem precisarmos de criar um caso isolado para o primeiro elemento.

2.2 Trechos de Código Explicados

A - [Jean Kenold]: Validação e Segurança de Dados

```
void menuCadastrarAgendamento(ListaAgendamentos *la, ListaPacientes *lp) {
    Agendamento a;
    printf("CPF do Paciente: ");
    scanf("%s", a.cpf_paciente);

    // Regra de Negócio: O paciente deve existir para agendar
    if (buscarPacienteCPF(lp, a.cpf_paciente) == NULL) {
        printf("Erro: Paciente nao encontrado no sistema.\n");
        return;
    }
    // ... segue para leitura de data e hora
}
```

Explicação: Este trecho demonstra a integração entre as duas listas lineares. Antes de alocar memória para um agendamento, o sistema realiza uma busca na ListaPacientes. Se o CPF não for encontrado, a função interrompe a execução (return), garantindo a integridade referencial dos dados da clínica e evitando agendamentos órfãos.

B- [Thales Cercal]: Responsável pela estruturação do lista.h e lista.c, desenvolvendo a lógica bruta de alocação de memória (funções malloc, inserção no início para pacientes e controle do header para agendamentos), verificacao de que tem memoria suficiente, uso do free, e tambem prototipagem geral, fazendo os outros headers.

```
ListaAgendamentos* criarListaAgendamentos() {
    ListaAgendamentos *lista = (ListaAgendamentos*)malloc(sizeof(ListaAgendamentos));
    if (lista == NULL) { printf("Erro de memoria\n"); exit(1); }

    lista->header = (NoAgendamento*)malloc(sizeof(NoAgendamento));
    if (lista->header == NULL) { printf("Erro de memoria\n"); exit(1); }

    lista->header->proximo = NULL;
    lista->tamanho = 0;
    return lista;
}
```

Explicação: Como responsável pela lógica bruta, implementei a verificação de segurança em cada malloc. Caso o sistema operacional não consiga fornecer memória, o programa é encerrado imediatamente para evitar acessos a ponteiros nulos. Além disso, inicializei o Nó Cabeçalho (sentinela), que é fundamental para que as funções de inserção e remoção no histórico funcionem de forma padronizada, sem casos isolados para a primeira posição.

C- [Pablo Ishiai]: Responsável pela criação do menu principal (main.c), módulo de pacientes (paciente.c) e formatação das saídas no terminal para melhor experiência do utilizador. Foi também quality assurance geral, buscando bugs na hora de execucao.

```
int main() {
    ListaPacientes *lp = criarListaPacientes();
    ListaAgendamentos *la = criarListaAgendamentos();
    int opcao;
    do {
        exibirMenu();
        if (scanf("%d", &opcao) != 1) {
            printf("Entrada invalida!\n");
            while(getchar() != '\n'); // Limpeza do buffer
            continue;
        }
        // ... switch case para chamar as funções
    } while (opcao != 0);
    destruirListaPacientes(lp); // QA: Garantindo liberação de memória
}
```

Explicação: Minha contribuição focou na experiência do utilizador e na qualidade do software. Implementei o loop principal que mantém o sistema vivo e a limpeza do buffer do teclado para evitar que o menu entrasse em loop infinito com entradas inválidas. Como parte do Quality Assurance, garanti que, ao encerrar o programa, as funções de destruição fossem chamadas para liberar todos os nós alocados, garantindo que o software termine sem vazamentos de memória (memory leaks).